



PONTIFICIA UNIVERSIDAD CATÓLICA DEL ECUADOR
SEDE MANABÍ · CARRERA DE INGENIERÍA EN SOFTWARE

Materia: Desarrollo de Sistemas de Información

Docente: Ing. José Luis Naranjo Villota

Nombre: Frank Marcelo Morales Moreira

Enlace del Repositorio: <https://github.com/castro-bot/portoSinFiltro.git>

Enlace de Producción (Despliegue): <https://porto-sin-filtro.vercel.app/>

Portoviejo, Manabí, Ecuador
Agosto de 2026



Reporte de Gestión e Historial Técnico

Durante el ciclo de desarrollo de PortoSinFiltro, gestionado bajo la metodología Scrum, mi participación técnica se centró en la integración de servicios de geolocalización, almacenamiento en la nube y aseguramiento de la calidad (QA). A continuación, se detallan los tickets principales atendidos en la herramienta de gestión (Jira) y los aportes reflejados en el repositorio Git:

1. ID de Ticket: PSF-15 - Implementación de Mapa Interactivo con Leaflet

- Descripción: Configuración e integración de la librería Leaflet en el frontend (React + Vite) para la visualización geoespacial de las denuncias ciudadanas. Se implementó la lógica para capturar las coordenadas (latitud y longitud) y renderizar marcadores precisos en el mapa de Portoviejo.
- Estado: Finalizado.
- Aporte en Git (Commits/PRs): feat: integracion mapa interactivo leaflet y captura de coordenadas.

2. ID de Ticket: PSF-22 - Configuración de Storage y Validación de Imágenes

- Descripción: Desarrollo del módulo de carga de evidencias fotográficas conectando el frontend con Supabase Storage. Se programaron validaciones estrictas de formato y tamaño de archivos en el cliente antes de la subida para evitar saturación y asegurar la integridad de la base de datos relacional.
- Estado: Finalizado.
- Aporte en Git (Commits/PRs): feat: conexion supabase storage y validacion de formatos de foto denuncia.

3. ID de Ticket: PSF-38 - Pruebas de Calidad (QA) y Entorno Standalone

- Descripción: Ejecución de pruebas de aseguramiento de calidad sobre el dashboard y los formatos de subida. Adicionalmente, se estructuró un "Plan B standalone" como contingencia para la demostración en vivo, garantizando que el flujo principal de carga y visualización funcione bajo escenarios de estrés o desconexión parcial de servicios.
- Estado: Finalizado.



- Aporte en Git (Commits/PRs): test: scripts de qa dashboard y configuracion plan b standalone.

1. Estrategia de validación y optimización de medios en el cliente: Durante la planificación del módulo de denuncias, identifiqué un posible cuello de botella en el ancho de banda y los costos de almacenamiento de Supabase Storage. Propuse e implementé una lógica de validación estricta de formatos fotográficos directamente en el frontend antes de realizar la petición HTTP. Esta idea técnica evitó que archivos corruptos o excesivamente pesados consumieran los recursos del servidor y generaran errores en tiempo de ejecución, mejorando significativamente la resiliencia del sistema y la experiencia del usuario (tiempos de carga menores en conexiones inestables).

2. Diseño del "Plan B Standalone" para alta disponibilidad en pruebas: Considerando los riesgos identificados de latencia o caídas de red durante las presentaciones técnicas (demostración en vivo), propuse al equipo estructurar un entorno o flujo de respaldo denominado "Plan B Standalone". Esta estrategia arquitectónica permitió aislar ciertos flujos críticos del frontend para que pudieran ser demostrados y testeados (QA) sin depender al 100% de respuestas síncronas de la base de datos de producción en cada interacción, garantizando una demostración fluida e ininterrumpida frente al cliente/docente.

Conclusión y Experiencia Personal

El desarrollo de PortoSinFiltro ha sido un reto técnico invaluable que me permitió consolidar mis conocimientos en arquitecturas modernas cliente-servidor, específicamente integrando React/Vite con Backend-as-a-Service (Supabase). A nivel personal, el mayor aprendizaje radicó en comprender la importancia de las metodologías ágiles y el aseguramiento de la calidad (QA) dentro de un entorno colaborativo. Comprobé que escribir código funcional no es suficiente; la verdadera ingeniería de software requiere anticiparse a los fallos, estructurar planes de contingencia para los despliegues en producción y coordinar de forma efectiva el trabajo en equipo a través del control de versiones. Esta experiencia me dota de una visión mucho más madura sobre el ciclo de vida real del software.